

Wake-Up (Start-Up) Procedure — v0.9 DRAFT

15 July 2026 · Drafted from the lessons of Sessions 1–5 (log entries cited per step) · Status: DRAFT — to be executed end-to-end, timed, and amended in Session 6, then issued as v1.0. Companion documents: Sleep (Shut-Down) Procedure (to be drafted); Session Planner; Pilot Synthesis §3 (standing doctrine).

1. Purpose and principle

This procedure brings the pilot workstation from an unknown overnight state to a verified working state before the experimental subject (Claude Code) is launched or tasked. The governing principle, paid for across five sessions: the condition you believe you are in and the one you are actually in diverge invisibly (Lesson 1); the machine changes while unattended (Lesson 4); therefore every session opens by verifying state against records, never against memory. Nothing touches the subject until every step below passes.

Timing note for Session 6: record the duration of each step in the margin during first full execution; the v1.0 issue will carry a total expected time and per-step norms.

2. Preconditions

The operator has: the previous session's log document and its recorded closing state (versions, session IDs, open items); the Session Planner current version; this procedure. If the previous session's sleep procedure was not executed (e.g. unplanned shutdown), note this at Step 1 and treat every check below as first-time discovery rather than confirmation.

3. The procedure

Step 1 — Open the log before the machine

Start the session entry in the pilot log: session number, date, operator, and the expected state carried forward from the previous session's close (subject version, model, workspace condition, open items). Every check below verifies against this record. (Doctrine: the chat is a convenience; the document is custody — Synthesis §3.)

Pass: A written expected-state exists before any command runs.

Step 2 — Build the bench first

Open a NEW PowerShell window from the Start menu — never reuse a window whose history is unknown, and never a window that has hosted Claude Code (Entries 058, 062: role-alternating windows caused every routing failure of the pilot). Retitle it and start its self-transcript immediately, so the bench's own record begins before its first check.

```
$Host.UI.RawUI.WindowTitle = "AUDIT BENCH - do not type tasks here"
Start-Transcript "E:\AAll XPlain\Dossier Space Pilot\ENRON Data Set Test 1\audit-bench-
transcript-sessionN-YYYY-MM-DD.txt" -Append
```

Pass: Title bar reads AUDIT BENCH; transcript confirmed started with its file path.

Step 3 — Park the bench outside the workspace

Set the bench's working directory to the exhibit level, never inside the evidence workspace (Entry 042: a prompt parked in the workspace turned a truncated command into evidence contamination; PowerShell defaults unresolved destinations to the current directory).

```
cd "E:\AAll XPlain\Dossier Space Pilot\ENRON Data Set Test 1"
```

Pass: Prompt line ends ...ENRON Data Set Test 1> — read it aloud.

Step 4 — Census the tool's own stores

The subject remembers through channels invisible at the prompt (Lesson 2), and its storage anatomy moves between versions (Entry 061). Check ALL known locations; anything new, reborn, or grown since the recorded close is a finding to resolve before launch — especially any memory folder (Entry 027 precedent: read, archive, delete, verify).

```
dir "C:\Users\ktomb\.claude" -Directory
dir "C:\Users\ktomb\.claude\projects\E--AA11-XPlain-Dossier-Space-Pilot-ENRON-Data-Set-Test-1-enron-mail-20150507" -Recurse -File
dir "C:\Users\ktomb\.claude\sessions"
dir "C:\Users\ktomb\.claude\file-history"
```

Pass: Top-level anatomy matches the recorded close (currently: backups, cache, downloads, file-history, paste-cache, plugins, projects, session-env, sessions); no memory folder; transcripts accounted for against the previous session's IDs. Deviations logged before proceeding.

Step 5 — Verify the evidence workspace

Full listing with hidden items — -Force is mandatory (Entry 044: the .git directory is invisible without it; Entry 042: foreign .jsonl files were the tell that exposed the pilot's worst boundary event).

```
dir "E:\AA11 XPlain\Dossier Space Pilot\ENRON Data Set Test 1\enron_mail_20150507" -Force
```

Pass: Contents match the recorded close exactly — currently: .git (hidden), maildir, pilot_outputs, .gitattributes, .ps1, allen-p.zip, CLAUDE.md, manifest.txt, Rename-EnronFilesToEml.ps1, _rename_eml_log.csv. No foreign files. pilot_outputs contents match the per-run archiving state recorded at close.

Step 6 — Verify the primer by identity, not impression

Confirm WHICH primer is deployed by size first, contents second (Entry 028: a read-back accepted on impression let the wrong primer govern for a full day — verification theatre). Current identities: v0.1 standing primer = 8,264 bytes; six-line safety primer = ~271 bytes. The size alone distinguishes them; the type check confirms.

```
dir "E:\AA11 XPlain\Dossier Space Pilot\ENRON Data Set Test 1\enron_mail_20150507\CLAUDE.md"
type "E:\AA11 XPlain\Dossier Space Pilot\ENRON Data Set Test 1\enron_mail_20150507\CLAUDE.md"
```

Pass: Size matches the primer the session's design requires; contents read (not skimmed) against the banked reference copy. A mismatch stops the session: the experimental condition does not exist until it is fixed.

Step 7 — Position the subject's window deliberately

Open (or repurpose) the subject's window; set its directory to the WORKSPACE — one level below the exhibits — and read the path aloud before anything else is typed (Entry 062: a launch from the exhibit level loaded no primer, disclosed experiment filenames, and voided a session; the trust prompt will NOT warn, because stale trust grants from Session 1 persist — trust does not expire, vigilance must).

```
cd "E:\AA11 XPlain\Dossier Space Pilot\ENRON Data Set Test 1\enron_mail_20150507"
```

Pass: Prompt line ends ...enron_mail_20150507> — read aloud. The window is confirmed as the subject channel: tasks and steering only, from here on.

Step 8 — Launch and record the subject's identity

Launch plain (no flags — Entry 063: --session-id fails silently and manufactures false provenance). Read the banner and record ALL THREE elements against the previous close: client version, model, workspace

path. Any drift is not an error but a measurement — log it, and if the model has changed generation, STOP and consult the Planner before any measured run (Entry 039/040 precedent: the comparison design must be consciously rebuilt, never silently continued).

claude

Pass: Banner read to the log: version, model, path. Drift logged. Session identity to be harvested from the tool's store or the exit banner (recorded as reality, not intention).

Step 9 — Verify the primer loaded — by question

Ask the subject what governs it. A correctly-floored primed session answers FROM CONTEXT, citing the standing primer, with no shell commands and no approval prompts (Entry 064). Shell-reaching on this question indicates a wrong floor or unloaded primer (Entry 062) — refuse any outward-reaching command, exit, and return to Step 7.

What primer governs this session?

Pass: Answer names and accurately summarises the deployed primer from loaded context; zero approval prompts triggered. For a baseline (six-line) session, the answer should reflect only the constraints — any richer governance description means the wrong primer is deployed.

Step 10 — Verify coldness — or accept warmth deliberately

For any measured run: confirm the session knows nothing it should not. Ask what it knows of prior work; a clean cold subject distinguishes what it can SEE on disk from what it REMEMBERS (nothing). First-person continuity with prior sessions (“as we did earlier”) means a warm session (Entry 025: continuity can be invisible — the display and the context can disagree) — exit and relaunch. Warm continuation is legitimate ONLY as a recorded condition, never as an accident.

What do you know about this workspace and any previous work done here?

Pass: Answer shows found-vs-remembered discipline; no unprompted references to prior runs, reports, or the pilot's framing beyond what the deployed primer itself discloses. Watch for and log any memory-writing during this check (Entry 027: asked what it remembered, the subject once created memories).

4. Completion

All ten steps passed: the session is open, the log records the verified state, and the subject may be tasked. Any step failed: the failure is the session's first entry, resolved per the cited precedent before work proceeds — the pilot's history says the failures are the findings.

Session 6 first-execution actions: time each step; note any command that errors or any check this draft omits; amend and issue as v1.0. Known open questions for v1.0: whether the frozen-subject configuration (auto-update, suggestions, memory disabled) changes Steps 4 and 8; whether the sessions/file-history paths in Step 4 remain current after the next tool update (Entry 061: the audit's instruments rot at the tool's release cadence — this procedure includes itself in that warning).

GPT Leonard Assurance

My overall assessment would be:

Aspect	Assessment
Operational clarity	9.5/10

Aspect	Assessment
Experimental integrity	10/10
Reproducibility	9/10
Custody thinking	10/10
Readability	8.5/10
Ready for GitHub	Yes, after minor edits

The document clearly reflects the lessons from the pilot rather than being a theoretical procedure. Almost every step is grounded in a specific lesson from Sessions 1–5, which is exactly what gives it authority.